Tarea M40 SQLite Javier Bueno Sanchez

August 14, 2026

1 Tarea M40 SQLite Javier Bueno Sánchez.

1.0.1 Creación y consulta de una base de datos SQLite con Python.

1.0.2 Objetivo.

Crear una base de datos SQLite mediante Python, seleccionar cinco clientes de un archivo CSV e insertarlos en una tabla denominada `Clientes`. Posteriormente, verificar la información almacenada mediante consultas realizadas en Python y DB Browser for SQLite, además de documentar las operaciones en un archivo SQL.

1.0.3 Índice

1. Importación de librerías.
2. Carga y revisión del archivo CSV.
3. Selección de cinco clientes.
4. Creación de la base de datos SQLite.
5. Creación de la tabla `Clientes`.
6. Inserción de los registros.
7. Consulta de los clientes almacenados.
8. Verificación en DB Browser for SQLite.
9. Generación del archivo `Operaciones.sql`.
10. Conclusiones.

1. Importación de librerías.

Para desarrollar la actividad se utilizarán las siguientes librerías:

- `pandas`: permite cargar, revisar y seleccionar los datos del archivo CSV.
- `sqlite3`: permite crear y consultar la base de datos SQLite desde Python.

[1]:
```
import pandas as pd
import sqlite3

print("Librerías importadas correctamente.")
```

```
Librerías importadas correctamente.
```

2. Carga y revisión del archivo CSV.

Se carga el archivo proporcionado para conocer su estructura, comprobar sus columnas y verificar la información antes de crear la base de datos.

[2]:
```
archivo_csv = "Cientifico de datos M40 Customers.csv"

clientes_csv = pd.read_csv(archivo_csv)

print("Dimensiones del archivo:", clientes_csv.shape)
display(clientes_csv.head())
```

```
Dimensiones del archivo: (91, 8)
```

	CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country	PurchaseUSD
0	1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany	14861
1	2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constituci?n 2222	M?xico D.F.	5021	Mexico	9900
2	3	Antonio Moreno Taquer?a	Antonio Moreno	Mataderos 2312	M?xico D.F.	5023	Mexico	7735
3	4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK	10295
4	5	Berglunds snabbk?p	Christina Berglund	Berguvsv?gen 8	Lule?	S-958 22	Sweden	6755

3. Selección de cinco clientes.

Para realizar la actividad se seleccionan los primeros cinco registros del archivo CSV mediante el método `head(5)`. Este criterio permite obtener una muestra objetiva y reproducible, sin modificar la información original.

[3]:
```
clientes_seleccionados = clientes_csv.head(5).copy()

print("Cantidad de clientes seleccionados:", len(clientes_seleccionados))
display(clientes_seleccionados)
```

```
Cantidad de clientes seleccionados: 5
```

	CustomerID	CustomerName	ContactName
0	1	Alfreds Futterkiste	Maria Anders
1	2	Ana Trujillo Emparedados y helados	Ana Trujillo
2	3	Antonio Moreno Taquer?a	Antonio Moreno

```
3           4                    Around the Horn        Thomas Hardy
4           5                  Berglunds snabbk?p  Christina Berglund

                        Address         City PostalCode  Country  PurchaseUSD
0                 Obere Str. 57       Berlin      12209  Germany        14861
1  Avda. de la Constituci?n 2222  M?xico D.F.       5021   Mexico         9900
2                 Mataderos 2312  M?xico D.F.       5023   Mexico         7735
3               120 Hanover Sq.       London    WA1 1DP       UK        10295
4                Berguvsv?gen 8        Lule?   S-958 22   Sweden         6755
```

4. Creación de la base de datos SQLite.

Se establece una conexión con SQLite mediante la función `sqlite3.connect()`. Si el archivo `Base.db` no existe, SQLite lo crea automáticamente en la misma carpeta donde se encuentra el notebook.

[4]:
```
nombre_base_datos = "Base.db"

conexion = sqlite3.connect(nombre_base_datos)

print(f"Base de datos '{nombre_base_datos}' creada o conectada correctamente.")
```

```
Base de datos 'Base.db' creada o conectada correctamente.
```

5. Creación de la tabla Clientes.

Se crea la tabla `Clientes` con una estructura compatible con los datos del archivo CSV. `CustomerID` se establece como llave primaria; los campos descriptivos se almacenan como texto y `PurchaseUSD` como número decimal.

[5]:
```
cursor = conexion.cursor()

cursor.execute("""CREATE TABLE IF NOT EXISTS Clientes (
    CustomerID INTEGER PRIMARY KEY,
    CustomerName TEXT NOT NULL,
    ContactName TEXT,
    Address TEXT,
    City TEXT,
    PostalCode TEXT,
    Country TEXT,
    PurchaseUSD REAL);""")

conexion.commit()

print("Tabla 'Clientes' creada correctamente.")
```

```
Tabla 'Clientes' creada correctamente.
```

6. Inserción de los registros.

Antes de insertar la información se eliminan los registros existentes de la tabla. Esto permite repetir la ejecución sin generar duplicados. Posteriormente, los cinco clientes seleccionados se incorporan

mediante una consulta parametrizada.

[6]:
```
cursor.execute("DELETE FROM Clientes;")

consulta_insercion = """INSERT INTO Clientes (
    CustomerID,
    CustomerName,
    ContactName,
    Address,
    City,
    PostalCode,
    Country,
    PurchaseUSD)VALUES (?, ?, ?, ?, ?, ?, ?, ?);"""

registros = list(
    clientes_seleccionados.itertuples(index=False, name=None))

cursor.executemany(consulta_insercion, registros)
conexion.commit()

print("Registros insertados correctamente:", cursor.rowcount)
```

```
Registros insertados correctamente: 5
```

7. Consulta de los clientes almacenados.

Se realiza una consulta general con `SELECT *` para extraer desde Python todos los registros guardados en la tabla `Clientes`. El resultado permite comprobar que la inserción fue correcta.

[7]:
```
consulta_clientes = """
SELECT *
FROM Clientes
ORDER BY CustomerID;
"""

clientes_base_datos = pd.read_sql_query(consulta_clientes, conexion)

print("Cantidad de registros recuperados:", len(clientes_base_datos))
display(clientes_base_datos)
```

```
Cantidad de registros recuperados: 5
```

	CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country	PurchaseUSD
0	1	Alfreds Futterkiste	Maria Anders					
1	2	Ana Trujillo Emparedados y helados	Ana Trujillo					
2	3	Antonio Moreno Taquer?a	Antonio Moreno					
3	4	Around the Horn	Thomas Hardy					
4	5	Berglunds snabbk?p	Christina Berglund					

```
0                    Obere Str. 57        Berlin      12209  Germany      14861.0
1  Avda. de la Constituci?n 2222  M?xico D.F.       5021   Mexico       9900.0
2                  Mataderos 2312  M?xico D.F.       5023   Mexico       7735.0
3                 120 Hanover Sq.        London    WA1 1DP       UK      10295.0
4                  Berguvsv?gen 8         Lule?   S-958 22   Sweden       6755.0
```

8. Verificación en DB Browser for SQLite.

Después de crear la base de datos, se abrió el archivo `Base.db` en DB Browser for SQLite. En la pestaña de estructura se confirmó la existencia de la tabla `Clientes` y sus ocho campos.

Posteriormente, se ejecutó la siguiente consulta general:

"'sql SELECT * FROM Clientes ORDER BY CustomerID;

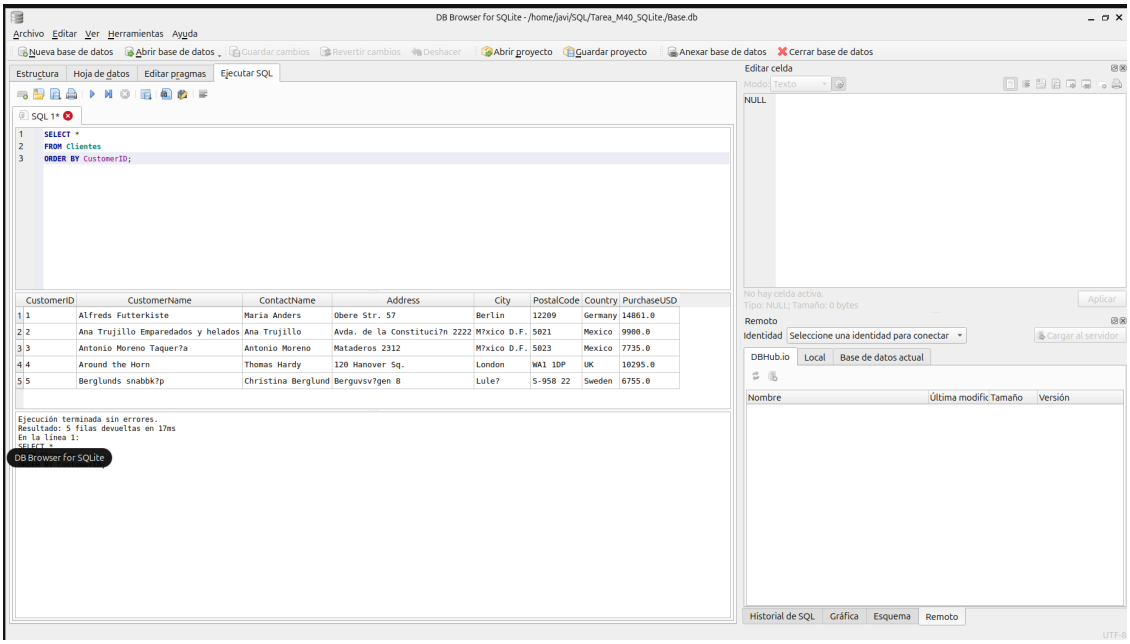
[8]:
```
conexion.close()

print("Conexión con la base de datos cerrada correctamente.")
```

```
Conexión con la base de datos cerrada correctamente.
```

Evidencia de la consulta en DB Browser for SQLite.

La siguiente captura confirma que la consulta devolvió los cinco registros almacenados en la tabla `Clientes`.



9. Generación del archivo Operaciones.sql.

Para documentar las operaciones realizadas, se genera un archivo SQL a partir de la base de datos. El archivo contiene las instrucciones necesarias para crear la tabla `Clientes` e insertar los cinco registros almacenados.

[9]:
```
conexion = sqlite3.connect("Base.db")

with open("Operaciones.sql", "w", encoding="utf-8") as archivo_sql:
    for instruccion in conexion.iterdump():
        archivo_sql.write(f"{instruccion}\n")

conexion.close()

print("Archivo 'Operaciones.sql' generado correctamente.")
```

```
Archivo 'Operaciones.sql' generado correctamente.
```

Verificación del archivo SQL.

Se revisa el contenido de `Operaciones.sql` para confirmar que incluye la estructura de la tabla y los registros necesarios para reconstruir la base de datos.

[10]:
```
with open("Operaciones.sql", "r", encoding="utf-8") as archivo_sql:
    contenido_sql = archivo_sql.read()

print(contenido_sql)
```

```
BEGIN TRANSACTION;
CREATE TABLE Clientes (
    CustomerID INTEGER PRIMARY KEY,
    CustomerName TEXT NOT NULL,
    ContactName TEXT,
    Address TEXT,
    City TEXT,
    PostalCode TEXT,
    Country TEXT,
    PurchaseUSD REAL);
INSERT INTO "Clientes" VALUES(1,'Alfreds Futterkiste','Maria Anders','Obere Str.
57','Berlin','12209','Germany',14861.0);
INSERT INTO "Clientes" VALUES(2,'Ana Trujillo Emparedados y helados','Ana
Trujillo','Avda. de la Constituci?n 2222','M?xico D.F.','5021','Mexico',9900.0);
INSERT INTO "Clientes" VALUES(3,'Antonio Moreno Taquer?a','Antonio
Moreno','Mataderos 2312','M?xico D.F.','5023','Mexico',7735.0);
INSERT INTO "Clientes" VALUES(4,'Around the Horn','Thomas Hardy','120 Hanover
Sq.','London','WA1 1DP','UK',10295.0);
INSERT INTO "Clientes" VALUES(5,'Berglunds snabbk?p','Christina
Berglund','Berguvsv?gen 8','Lule?','S-958 22','Sweden',6755.0);
COMMIT;
```

10. Conclusiones.

La actividad permitió crear correctamente la base de datos `Base.db` mediante Python y SQLite. El archivo CSV fue revisado previamente y se identificaron 91 registros con ocho columnas, de los cuales se seleccionaron los primeros cinco clientes mediante un criterio reproducible.

Los registros fueron almacenados en la tabla `Clientes`, utilizando tipos de datos adecuados para cada campo. La consulta realizada desde Python recuperó los cinco clientes sin errores y la verificación independiente en DB Browser for SQLite confirmó tanto la estructura de la tabla como la información almacenada.

Finalmente, se generó el archivo `Operaciones.sql`, que contiene las instrucciones necesarias para crear la tabla e insertar los registros. Con ello se cumplieron los requerimientos de creación, consulta, verificación y documentación de la base de datos.